

DAA-0619

CO3-AT1 Experiments-Based Assessment

Title

Analysis of Recursion Elimination Techniques in Merge Sort.

Objective

To analyse the effect of eliminating recursion in Merge Sort by converting recursive calls into an iterative process. The comparison is based on execution time, memory usage, performance, and practicality.

Algorithm

Recursive Merge Sort

1. Divide the array into two halves.
2. Recursively sort both halves.
3. Merge the sorted halves.
4. Continue until the array is sorted.

Iterative Merge Sort

1. Treat each element as a sorted subarray.
2. Merge adjacent subarrays of size 1.
3. Double the subarray size after each pass.
4. Continue until the entire array is sorted.

Analysis:

Parameter	Recursive Merge Sort	Iterative Merge Sort
Execution Time	Slightly higher	Slightly Lower
Memory Usage	Higher	Lower
Function Calls	Yes	No
Stack Usage	Uses Recursion Stack	No stack required
Risk of Stack Overflow	Possible	No
Practically	Easy to Understand	Better for large datasets

Time Complexity:

Case	Complexity
Best	$O(n \log n)$
Average	$O(n \log n)$
Worst	$O(n \log n)$

Performance Interpretation

- Recursive Merge Sort is simpler to implement but requires additional memory for recursive function calls.
- Iterative Merge Sort removes recursion, reducing stack memory usage.
- Execution time is generally slightly better in the iterative version because function call overhead is eliminated.
- Both algorithms maintain the same sorting efficiency of $O(n \log n)$.

Source Code:

```
1 import time
2 import tracemalloc
3 # Merge Function
4 def merge(left, right):
5     result = []
6     i = 0
7     j = 0
8     while i < len(left) and j < len(right):
9         if left[i] <= right[j]:
10             result.append(left[i])
11             i += 1
12         else:
13             result.append(right[j])
14             j += 1
15     result.extend(left[i:])
16     result.extend(right[j:])
17     return result
18 def merge_sort_recursive(arr):
19     if len(arr) <= 1:
20         return arr
21     mid = len(arr) // 2
22     left = merge_sort_recursive(arr[:mid])
23     right = merge_sort_recursive(arr[mid:])
24     return merge(left, right)
25 def merge_sort_iterative(arr):
26     n = len(arr)
27     size = 1
28     while size < n:
29         left = 0
30         while left < n:
31             mid = min(left + size, n)
32             right = min(left + 2 * size, n)
33             merged = merge(arr[left:mid], arr[mid:right])
34             arr[left:right] = merged
35             left += 2 * size
36         size *= 2
37     return arr
38 # ----- Input -----
39 n = int(input("Enter the number of elements: "))
40 data = []
41 print("Enter the elements:")
42 for i in range(n):
43     data.append(int(input()))
44
45 # ----- Recursive -----
46 arr1 = data.copy()
47 tracemalloc.start()
48 start = time.perf_counter()
49 sorted_recursive = merge_sort_recursive(arr1)
50 end = time.perf_counter()
51 current, peak = tracemalloc.get_traced_memory()
52 tracemalloc.stop()
53 recursive_time = end - start
54 recursive_memory = peak / 1024
55 # ----- Iterative -----
56 arr2 = data.copy()
57 tracemalloc.start()
58 start = time.perf_counter()
59 sorted_iterative = merge_sort_iterative(arr2)
60 end = time.perf_counter()
61 current, peak = tracemalloc.get_traced_memory()
62 tracemalloc.stop()
63 iterative_time = end - start
64 iterative_memory = peak / 1024
65 # ----- Output -----
66 print("\nRecursive Merge Sort")
67 print("Sorted Array :", sorted_recursive)
68 print("Execution Time :", recursive_time, "seconds")
69 print("Peak Memory :", recursive_memory, "KB")
70 print("\nIterative Merge Sort")
71 print("Sorted Array :", sorted_iterative)
72 print("Execution Time :", iterative_time, "seconds")
73 print("Peak Memory :", iterative_memory, "KB")
```

Output:

```
Enter the number of elements: 10
Enter the elements:
23
3334
6
12
23
6
89
1
26
30

Recursive Merge Sort
Sorted Array : [1, 6, 6, 12, 23, 23, 26, 30, 89, 3334]
Execution Time : 9.839795529842377e-05 seconds
Peak Memory : 0.2578125 KB

Iterative Merge Sort
Sorted Array : [1, 6, 6, 12, 23, 23, 26, 30, 89, 3334]
Execution Time : 7.700896821916103e-05 seconds
Peak Memory : 0.2578125 KB

...Program finished with exit code 0
```